FH University of Applied Sciences TECHNIKUM WIEN

BACHELOR PAPER

Thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science in Engineering at the University of Applied Sciences Technikum Wien - Degree Program XXXXX

Development of a root-state and child-state q-table abstraction algorithm for the game 2048

By: Stefan Werner
Student Number: 2310257227

Supervisor 1: [Titel noch hinzufügen] Marius Führer

Vienna, 10.04

Declaration of Authenticity

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz/ Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and according to the rules currently applicable at the UAS Technikum Wien and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.”

10.04

Place, Date

Digital Signature

Kurzfassung

Diese Arbeit untersucht eine Methode zur Reduktion der Speicheranforderungen von Q-Tabellen am Beispiel des Spiels 2048. Dabei wird die Q-Tabelle in zwei Teile aufgeteilt: eine Parent-Liste, die vollständige State-Action-Paare enthält, und eine Child-Liste, die lediglich Verweise auf Parent-Zustände sowie die notwendigen Matrixoperationen zur Ableitung speichert. Die zentrale Idee besteht darin, strukturell ähnliche Zustände beispielsweise durch Symmetrie, Rotation oder Skalierung gemeinsam zu repräsentieren. Anstatt für jeden Zustand eigene Q-Werte zu speichern, werden diese von einem entsprechenden Parent-Zustand übernommen und durch Transformation angepasst. Durch diesen Ansatz kann die Größe der Q-Tabelle deutlich reduziert werden, ohne die Entscheidungsqualität des Agenten wesentlich zu beeinträchtigen.

Schlagwörter: Reinforcement Learning, 2048, Q-Table, Abstraktion, Machine Learning, State Abstraktion, Matrix

Abstract

This work investigates a method for reducing the memory requirements of Q-tables using the game 2048 as an example. The Q-table is divided into two parts: a parent list containing complete state-action pairs, and a child list that stores only references to parent states and the necessary matrix operations for derivation. The central idea is to represent structurally similar states together, for example, through symmetry, rotation, or scaling. Instead of storing separate Q-values for each state, these are inherited from a corresponding parent state and adapted through transformation. This approach significantly reduces the size of the Q-table without substantially impairing the agent's decision quality.

Keywords: Reinforcement Learning, 2048, Q-Table, Abstraction, Machine Learning, State Abstraction, Matrix

Acknowledgements

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Maecenas porttitor congue massa. Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna.
Nunc viverra imperdiet enim. Fusce est. Vivamus a tellus.
Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Proin pharetra nonummy pede. Mauris et orci.
Aenean nec lorem. In porttitor. Donec laoreet nonummy augue.

Table of Contents

1 Introduction 6
1.1 Motivation 6
1.2 Problem Statement 6
1.3 Research Questions 7
1.4 Structure of the Thesis 7

2 Notation 7
2.1 Reinforcement Learning Notation 7
2.2 Abstraction Notation 9

3 Introduction to 2048 9
3.1.1.1 Example after the player chooses to move right: 10
3.2 Connection to Reinforcement Learning 10
3.2.1.1 State space 10
3.3 Connection to Q-table Abstraction 11

4 Introduction to Reinforcement Learning (RL) 12
4.1 Markov Decision process 13
4.2 Q-Learning 14
4.3 Q-Table Representation 15

5 Parent / Child Theory 16
5.1 Mathematical Definition 16
5.2 Transformation-Based state Equivalence 17
5.2.1 Rotation (r) 18
5.2.2 Scaling (doubling) (d) 18
5.2.3 Edge expansion (e) 19
5.2.4 Mirroring (m) 19
5.3 Parent-Child Representation 20
5.3.1.1 Parent: 20
5.3.1.2 Child: 21
5.3.1.3 Splitting 21
5.3.1.4 Reconstruction 22

6 Environment 23

7 Python Implementation 24
7.1 Main 24
7.2 Environment 24
7.2.1 Core Interaction 24
7.2.2 Reward Function 24
7.3 Agent 25
7.3.1 Q-Table Representation 25
7.3.2 Action Selection 26
7.3.3 Learning Process 26
7.3.4 Training Loop 27
7.3.5 Q-Table Storage 27
7.4 Operations 28
7.4.1 Transformation Operations 28
7.4.2 Action Transformation 29
7.4.3 PreCalculator 29
7.4.4 Parent-Child Abstraction 30
7.4.5 Reconstruction 30

8 Concept 31

9 Experiment and ResultsBibliography 31

List of Figures 33

List of Tables 34

Es konnten keine Einträge für ein Abbildungsverzeichnis gefunden werden.List of Abbreviations 34

Documentation table of AI-based tools 36

A: Heading of Appendix A 37

B: Heading of Appendix B 38

1 Introduction

Reinforcement Learning (RL) has become one of the most widely used approaches for solving decision making problems [2]. Especially for games or environments where modelling the entire Environment is difficult or impossible such as self-driving cars [3]. One of the most fundamental methods or learning structures for RL is called Q-learningt [1]. The agent learns connections between states and actions by storing the best move in a Q-Table. Q-learning works great for small environments where the state space is minor but suffers from a major limitation, the size of the Q-table grows increasingly with the number of different states [4]. This can lead to significant memory requirements and inefficiencies in storing this large number of states.

This thesis focuses on the game 2048 as an example of increasing state space size due to its exponential state growth [5]. Even small versions of the game with a smaller grid can lead to a large number of possible states. To address this issue, this thesis introduces a abstraction approach based on a parent child concept [5]. The core idea is the reduction of saved space by not saving every state but only saving root or in this thesis also called parent states and their corresponding child states. Child states are determined though matrix operations such as rotation doubling and edge detection. [6] [7]

1.1 Motivation

The Q-table is essentially the brain of the agent. It stores the connection between state and action, creating a state-action pair and directly determines the behaviour of the agent during the decision-making process. As previously stated in small environments this approach is efficient and easy to implement. But in more complex environments or non-deterministic environments this can easily lead to millions of state-action pairs with very similar states and actions. This becomes particularly problematic with scaling as a larger grid in a game like 2048 increases the state space dramatically [4]. A 4 by 4 2048 Q-table can easily exceed 10 million states (~ 1 GB). This approach would reduce the number of saved states by half [8] [9].

Many of these states are essentially the same just rotated or doubled and lead to wasted space which could be used to save more important states that are underrepresented in the Q-Table [6] [8] [9].

1.2 Problem Statement

The problem addressed in this thesis is the inefficient storage of Q-table entries in environments with large state spaces.

Traditional in Q-learning every state-action pair is stored separately regardless of the similarities between the pairs. This leads to redundant information being stored multiple times, increasing memory usage without adding new knowledge.
The idea is to design a method that reduces the number of stored states, avoids loss of information and allows reconstruction of the original Q-table when needed.

In particular, the problem can be formulated as follows:
How can a Q-table be compressed by identifying relationships between states, such that redundant states are represented through transformations of a smaller set of base states, without significantly degrading the learned policy?

1.3 Research Questions

1. How can similarities between states in the game 2048 be identified and represented?
2. Can a Q-table be compressed by representing states as children of other states through matrix transformation?
3. How does this abstraction approach effect the performance of the agent?
4. What is the trade of or overhead of this approach?
5. How does transformation depth affect compression ratio and policy accuracy?
6. What are the limitations of this approach and what can not be done?

1.4 Structure of the Thesis

This paper is divided into three parts:

- The First introduces reinforcement learning, including Markov decision processes and Q-learning. This section should provide the necessary background needed to understand the learning process and the limitations.
- The second part focuses on the concept of Q-table abstraction. It introduces the proposed root-state and child-state model.
- The third part presents the evaluation of the proposed method. It includes the comparison between origin and reconstructed Q-table.

2 Notation

2.1 Reinforcement Learning Notation

RL often uses a standardized notation. Here, s denotes a state, a an action, and r the reward received. The policy π describes the agent's decision rule. The following notation is often used in Reinforcement Learning [2].

s	$state$
s_t	$state\ at\ time\ t$
s'	$next\ state$
a	$action$
a_t	$action\ at\ time\ t$
a'	$next\ action$
t	$timestep/repetition$
r	$reward$
r_t	$reward\ at\ time\ t$
R	$total\ Reward$
e	$episodes$
γ	$discount\ Factor$
ϵ	$exploration\ rate$
$1-\epsilon$	$exploitation\ rate\ (probability\ of\ using\ the\ q-table)$
α	$learning\ rate$
S	$set\ of\ all\ states$
A	$set\ of\ all\ actions$
$A(s)$	$set\ of\ all\ possible\ actions\ given\ state\ s$
$R(s,a,s')$	$reward\ funtion$
G_t	$return\ (cumulative\ discounted\ reward)$
$P(s'\|s,a)$	$transition\ probability$
π	$policy, decision\ making\ rule$
$\pi(s)$	$action\ taken\ given\ state\ s\ under\ deterministic\ policy\ \pi$
$\pi(a\|s)$	$probability\ of\ taking\ action\ a\ given\ state\ s\ under\ stochastic\ policy\ \pi$
$\pi^*(s)$	$optimal\ policy, best\ action\ with\ highest\ expected\ return$
$V^\pi(s)$	$state\ value\ function, expected\ highest\ return\ given\ state\ s\ under\ policy\ \pi$
$V^*(s)$	$state\ value\ function\ (optimal\ policy), expected\ highest\ return\ given\ state\ s$ $under\ optimal\ policy\ \pi^*$
$Q^\pi(s,a)$	$action\ value\ function\ (\ Q-Function\ ), expected\ highest\ return$ $given\ action\ a\ in\ state\ s\ under\ policy\ \pi$
$Q^*(s,a)$	$action\ value\ function\ (\ Q-Function\ ), expected\ highest\ return$

$given\ action\ a\ in\ state\ s\ under\ optimal\ policy\ \pi^*$

[2]

2.2 Abstraction Notation

The following notation is specifically used for this bachelor's thesis and describes the relationship between root and child states.

p	$root/parent\ state$
c	$child\ state$
o	$operation$
P	$set\ of\ all\ \ root/parent\ states$
C	$set\ of\ all\ child\ states$
$C(p)$	$set\ of\ all\ child\ states\ for\ \ root/parent\ \ state\ p$
$\mathcal{O}$	$set\ of\ all\ operations$
O	$composition\ of\ operations$
$O(c)$	$set\ of\ all\ possible\ operations\ for\ child\ state\ c$
$O(p)$	$set\ of\ all\ possible\ operations\ for\ parent\ state\ p$
M	$memory\ usage\ of\ non\ abstracted\ Q-table$
M'	$memory\ usage\ of\ abstracted\ Q-table$
$M(n)$	$memory\ usage\ of\ non\ abstracted\ Q-table\ for\ grid\ of\ size\ n$
$M'(n)$	$memory\ usage\ of\ abstracted\ Q-table\ for\ grid\ of\ size\ n$
T	$time\ usage\ of\ non\ abstracted\ Q-table$
T'	$time\ usage\ of\ abstracted\ Q-table$
$T(n)$	$time\ usage\ of\ non\ abstracted\ Q-table\ for\ grid\ of\ size\ n$
$T'(n)$	$time\ usage\ of\ abstracted\ Q-table\ for\ grid\ of\ size\ n$

3 Introduction to 2048

The game 2048 was created by Gabriele Cirulli on march 2014. It is a single player, grid based puzzle game were the aim is to achieve the highest score possible by combining tiles with the same value into tiles of the combined value. It is played on a 4 by 4 matrix where every tile

starts with the value 0. Upon starting the game two random tiles will become tiles of value 2 with a ratio of 9/10 or a tile of value 4 with a chance of 1/10.

The player has 4 actions available from the start, up, down, left and right. Upon choosing an every tile will travel to the furthest possible tile in the given direction. A tile can have the value of 0 and a multiple of 2. For example a tile can be 0, 2, 4, 8, 16, 32, etc… A tile will stop when it hits the wall or another tile with a value other than 0 and its own value. For example a tile of value 4 will travers past tiles of value 0 but will stop upon reaching a tile of value 2 or larger. When the tile of value 4 reaches another tile of value 4 both tiles will combine into a tile of value 8.

3.1.1.1 Example after the player chooses to move right:

0	0	0	2
0	2	2	0
0	4	0	8
0	0	0	0

Action: Right

0	0	0	2
0	0	0	4
0	0	4	8
2	0	0	0

Figure 1: Example player input right tile shift

After every move a new tile is created again with a chance of being 2 or 4 at a random location.

3.2 Connection to Reinforcement Learning

The simplicity of the game leads to it being commonly used for reinforcement learning. Each player input produces a new state s. In context of RL the agent chooses an action a from the set of possible actions given the state [5] [2]. This interaction naturally forms a Markov Decision Process [6]. A challenge in applying reinforcement learning to 2048 is the vast amount of state space [5] [4]. The number of possible grid configurations grows exponentially due to the different tile values and their positions [11]. As a result, storing every state and every action in a Q-table becomes memory expensive very quickly. After running an agent on the game for just one hour can produce a Q-table with millions of entries and the size of gigabyte if not terabyte [4].

3.2.1.1 State space

The state space S of the game 2048 consists of all possible formations of the grid world [5]. As the game is played on a grid of size n by n where each tile can have a value between 0 and 2^k where k is the maximum tile value observed during the training, the state space can be defined as:

$$s \in S = \{0,2,4,8, \dots, 2^k\}^{n*n}$$

Each entry $s_{i,j}$ represents the tile at position (i, j).
In the implementation each state is represented as a flattened matrix:

$$s = (s_1, s_2, \dots, s_{n^2})$$

The theoretical size of the entire state space would be defined as:

$$|S| = (k+1)^{n^2}$$

But a lot of these states can not be reached due to the game rules or structural equivalents which motivates the abstraction approach discussed in this thesis.

3.3 Connection to Q-table Abstraction

This makes 2048 an ideal environment for this thesis. The idea is to not store millions of states as state action pairs but determine one parent state for many child states [9]. A parent p is a state that given the correct operations O can create any child state of the parent $|C(p)|$.

Many states share structural similarities but are counted as distinct states. A state s is a child state c of a parent state p if operations such as rotation, mirroring, doubling of values and scaling can create a copy of either the parent state or the child state [8] [12].

Many states in S exhibit structural similarities [8] [12]. This leads to the abstraction function [9]:

$$f: S \rightarrow P$$

Where $P \subseteq S$ is the set of parent states
This function maps a states s (child state c) to a corresponding parent state p such that either:

- $s = p$, if no parent state exists or
- There exists at least one operation that if applied to the parent p creates the child c

$$\exists\, O = \in \mathcal{O} : s = O(p)$$

The game 2048, with its grid based environment and the structured use of values that are powers of two, is particularly well suited for this approach [5]. The consistent scaling of tile values and the inherent symmetries within the grid enable the application of transformation-based abstractions [8] [12].

In this thesis the game will be simulated through 3 different game version. A 2 by 2 tile grid version, chosen for the purpose of fast testing of ideas and operations. A 3 by 3 version designed to find the effect of scaling operations and the finale 4 by 4 grid version where both ideas will be put together. In this version the goal consists of first reaching the tile 2048 and secondly seeing how far the agent is able to go.

4 Introduction to Reinforcement Learning (RL)

Machine Learning (ML) is a way for computers to understand data and or environment and applying specific actions given a certain input. ML divides itself into 3 sub-categories, Supervised learning where developers use labelled data to train A.I. models for predictive tasks. The data usually gets labelled by the developers themselves. The data is split into a training set and a test set and the A.I. is trained. In unsupervised learning the data is unlabelled and the A.I. is forced to find patterns or clusters in the data. The result is harder to validate and the way the A.I. found the answer can be hard for developers to understand such as in neural networks.

Other than in supervised learning and in unsupervised learning in reinforcement learning (RL) the A.I. or in RL often called agent learns not through data but through an environment, often simplified, through trial and error. Guided only with rewards and penalties the agent will find the best way through a maze or get the highest score in a game. It is often used in robotics or autonomous systems such as self-driving cars where agents must adapt over time. In these circumstances the agent chooses the best possible path with help of deep learning to avoid accidents on the road.

The environment returns a state s_t to the agent. The agent selects an action a_t via the given policy. The environment reacts to the action and updates the state to s_{t+1}. A reward r_{t+1} is returned to the agent. Based on the reward the agent adjusts the policy / Q-table. A new loop begins.

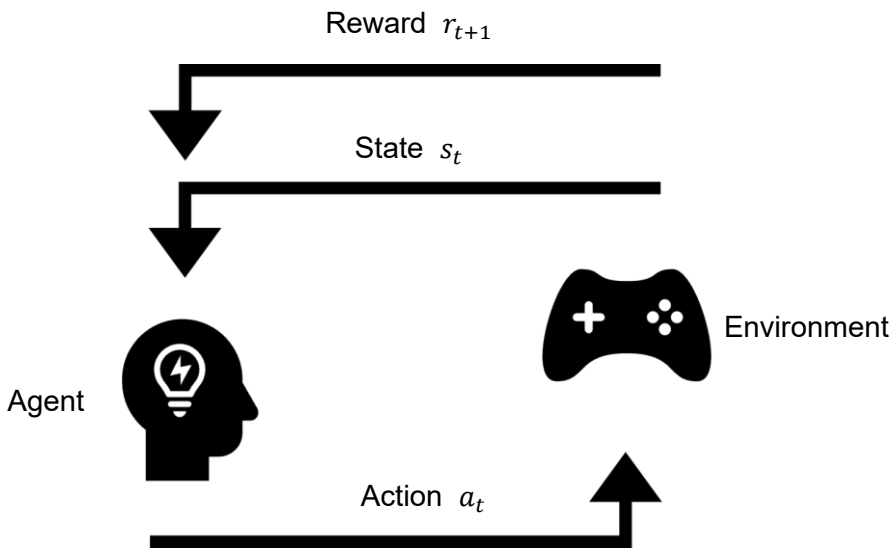

Figure 2: Environment Agent loop

In this part of the thesis the concepts and procedures of reinforcement learning will be discussed and its uses for this thesis.

4.1 Markov Decision process

The idea behind RL builds on the Markov Decision Process (MDP) The MDP defines the environment, while the policy determines which action to take in each state.

The chain begins in state s_0. The agent observes the state and selects an action a_0. In the case of the game 2048 an action can be any of the 4 cardinal directions up, down, left and right. This action is send to the environment and the environment changes based on the action. This leads to the state s_1. A reward r_1 is sent back to the agent to help him determent the next move. The agent observes the state again and chooses an action. This leads to the Markov decision chain:

$$s_t \rightarrow a_t \rightarrow r_{t+1} \rightarrow s_{t+1} \rightarrow \cdots$$

A key concept of the MDP is that each state only depends on the immediate state and action before it and not on every state before it. This can be derived from the transition probability:

$$P(s_{t+1}|s_t, a_t)$$

The objective of an RL agent is to learn a policy π which maximizes the expected sum of possible returns:

$$G_t = \sum_{k=0}^{e} \gamma^k r_t + k + 1$$

In the context of 2048 e is the number of episodes the agent is playing which can range from 1 to 100.000. In normal MDP it is assumed that e is ∞. The discount factor γ is a number between of 0 to 1 (starting with 1) and ensures that the agent prioritizes exploration first and exploitation later.

4.2 Q-Learning

Q-Learning is a value-based RL algorithm. Its aim is to learn the optimal action-value function $Q^*(s, a)$. This function returns the maximum expected sum of rewards that can be achieved by taking action a in state s and following the optimal policy. Q-Learning is the idea of training an agent by updating its brain over a specific length of time or episodes e. The brain is the Q-table the agent forms throughout the training. The first moves the agent does are random due to the value epsilon ε. The agent moves in random directions and marks down the state and the reward it receives from the environment. The agent updates the Q-table and transitions to the next state. In code this loop is done through the step() function. The Q-table is updated iteratively using the following update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

γ is the discount factor which controls the importance of future rewards. It is a value between 0 and 1 and is updated after every loop. A value of 1 results in more exploration while a value of 0 results in only exploitation. Exploitation in this case results in the agent only choosing the best action from the Q-table while exploration results in the agent choosing random paths.

α is the learning rate. It controls the importance of new values compared to old values. A high learning rate results in new values overriding old values and a low learning rate leads to old values being preferred.

r is the immediate reward. It is the reward the agent receives immediately after performing action a

$\max_{a'} Q(s', a')$ represents the estimated optimal future value

Q-Learning is an off-policy algorithm, it learns the optimal policy independently of the agents behavior. Depending on the exploration strategies the agent follows different approaches. In an ε-greedy approach the agent prioritizes exploration to explore the environment.

In the context of 2048 the agent receives the state as a numpy matrix. It observes the matrix and selects depending on the strategy one of the four possible actions. After each move it receives a reward based on the value of tiles merged by the chosen action. The agent updates the Q-table. By continuously updating the Q-values, the agent learns which actions lead to higher rewards and gradually improves its performance.

4.3 Q-Table Representation

A Q-table represents the brain of the Agent. It consists of state-actions pairs where each state is linked to a set of actions and their corresponding Q-values. The Q-values are an estimation of the reward the agent can expect when following each action. During training the agent updates these values based on the rewards received from the environment. This will lead to it gradually improving the policy its learning on. Eventually coming close to the optimal policy.

In the context of 2048 a state is a flattened 4 by 4 matrix or depending on the environment a 2 by 2 or 3 by 3 matrix. The actions correspond to the possible movements up, down, left and right.

$$(state, actionUP, actionDOWN, actionLEFT, actionRIGHT)$$

Over time the Q-table stores the learned evaluation of these actions. The agent then chooses the action with the best future results.

```
state;up;down;left;right
2,2,0,0;0.0;31.465654068747885;26.9191467740004;26.28482026848514
0,2,0,2;29.315083246115393;30.85967688016068;28.237153342637573;0.0
0,0,2,2;28.657980531401954;0.0;29.300089975361765;30.612445916335
2,0,0,2;29.307012062855954;29.25859113811383;28.59376732648875;31.468805139022386
0,2,2,0;28.855932552441022;31.218279534849756;28.152230173579934;29.538881587145653
2,0,2,0;26.2396816447543;25.86272150450492;0.0;31.870140204435806
4,0,0,2;18.096422119948237;21.524150639931975;24.67407151978731;29.75600496408652
```

Figure 3: normal Q-table representation

5 Parent / Child Theory

This thesis introduces a Q-table abstraction based on a parent-child representation of states. In a normal Q-learning setup for the game 2048, the Q-table grows exponentially. This leads to high memory consumption and inefficient storage. The core idea of this work is to reduce the number of stored states by identifying structural similarities between them. Instead of saving every state individually states are grouped into equivalence classes based on matrix transformations. Within each group a parent state represents the best action for all other states. This parent state is stored explicitly while all other states are saved with a key to the parent, the matrix operation and the previously best direction for the state.

5.1 Mathematical Definition

Defining this behavior mathematically. The set of parent states P is defined as a subset of the set of all states S:

$$P \subseteq S$$

A state s is classified as a parent state p if it cannot be generated from any other distinct state using a sequence of allowed operations and if p is an element of all the states S.

$$p \in P \;\Leftrightarrow\; \nexists s \in S, s \neq p, \exists O: p = O(s)$$

The set of child states C is defined as a subset of the set of all states S.

$$C \subseteq S$$

Every child state c in the set of all child states C is classified as a child state if and only if there exists exactly one parent state p in the set of all parent states P and a sequence of transformation operations that generate a child state c from the parent state p, where $c \neq p$.

$$c \in C \Leftrightarrow \exists!\, p \in P, \exists O: c = O(p), c \neq p$$

The way p is chosen is by sorting the Q-table by number of zeros in it and choosing the first matching parent in the sorted order.

$$z(s) = number\ of\ zero\ entries\ in\ s$$
$$s_i \prec s_j \;\Leftrightarrow z(s_i) > z(s_j)$$

$$f(s_i) = \begin{cases} p_k, & where\ p_k = \min\{p \in P | \exists O: s_i = O(p)\} \\ s_i, & otherwise \end{cases}$$

5.2 Transformation-Based state Equivalence

Two states are considered equivalent if one can be transformed into the other using a sequence of predefined matrix operations. Formally a state s can be a child state c of a parent state p if there exists at least one sequence of operations such that:

$$s_1 \sim s_2 \Leftrightarrow \exists O: s_1 = O(s_2)$$

$$c = O(p)$$

A sequence of operations can be:

$$r$$
$$ddr$$
$$mrdre$$

A Sequence of operations is defined as:

$$O = (o_1, o_2, \dots, o_k)$$

Where o_i is an element of the set of all possible operations:

$$o_i \in \mathcal{O}$$

A sequence is applied to a parent in an additive loop to generate a child state:

$$c = O(p) = r(d(r(p)))$$

Or mathematically:

$$c = O(p) = o_k(\dots o_1(p) \dots)$$

In order for this to work the actions must be able to be either transformed or must have the same value if a state is scaled to not lose context for the agent.

The actions of a state that is transformed must be of the same context for the agent to not lose information. If an action with a transformation applied reduces the context of a state for the agent the abstraction is a hindrance.

Information is defined as data that the agent uses to play the game. Information can be a state, an action or the hidden relation between states. The agent gets punished for choosing moves that result in high rewards in the early game just to lose in the long game. The agent will therefore sometimes choose actions that appear bad in the short term but add in the long term. This information should not get lost. The direction column in the child Q-table is designed to save the best direction not in the short term but in the long term and will transform the actions of the agent if the actions provided from the parent collide with the best direction of the original state.

The following transformation operations are considered when deciding between parent and child:

5.2.1 Rotation (r)

A state can be rotated by 90° increments. Two states are equivalent if one can be derived from the other if a rotation is applied. Matrix A and matrix B are the same if you can rotate A to get to B. This operation exploits the symmetry of grid based worlds as the action up can also be rotated to the left, down or right.

A

2	0	0	0
8	4	2	0
8	8	4	0
16	8	8	2

+ Rotation =

B

0	0	0	2
0	2	4	8
0	4	8	8
2	8	8	16

Figure 4: Transformation-Based Rotation

5.2.2 Scaling (doubling) (d)

A state can be scaled by multiplying all tile values by a constant factor. Two states are equivalent if one is a scaled value of the other. Matrix A and matrix B are the same if you can double

A to get to B. This is particulary effective in games like 2048 due to the exponential structure of the tile values.

A

2	0	0	0
8	4	2	0
8	8	4	0
16	8	8	2

+ Double =

B

4	0	0	0
16	8	4	0
16	16	8	0
32	16	16	4

Figure 5: Transformation-Based Scaling

5.2.3 Edge expansion (e)

States can be extended by adding empty or low-value tiles at the borders while preserving the internal structure. This allows smaller patterns to represent larger states. Matrix A and matrix B are the same if you can add an edge to A to get to B.

A

2	0	0	0
8	4	2	0
8	8	4	0
16	8	8	2

+ Edge =

B

2	2	2	0
8	4	2	2
8	8	4	2
16	8	8	2

Figure 6: Transformation-Based edge expansion

5.2.4 Mirroring (m)

A state can be mirrored along horizontal or vertical axes. Two states a equivalent if one is a mirrored version of the other. Matrix A and matrix B are the same if you can mirror A to get to B.

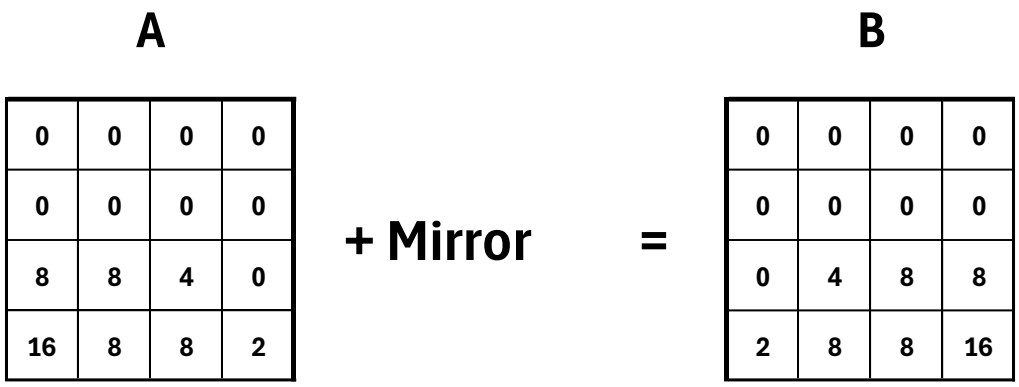

Figure 7: Transformation-Based Mirroring

5.3 Parent-Child Representation

Using the operations above the original Q-table can be divided into two components. A Q-table filled with parent state-action pairs:

$$(index, state, actionUP, actionDOWN, actionLEFT, actionRIGHT)$$

The index is used for the child to link to the parent. The state is a flattened matrix. The actions represent a float value of each direction on the grid chosen by the agent.

$$(1, \text{"0,0,2,0,2,4,2,4,8"}, 23.34, 13.3, 10.534, 32.5758)$$

The second part of the divided Q-table is the child Q-table filled with the index of the parent and operations applied to the parent to get to the original child.

$$(parentIndex, operations, direction)$$

The index is a key that points to the original parent and the operations are a string of operations applied to the child to get to the parent. The direction property insures the agent doesn't lose context of the state.

$$(12, \text{"rdde"}, 1)$$

This idea leads to 2 files the parent Q-table and the child Q-table.

5.3.1.1 Parent:

```
index;state;up;down;left;right
0;2,2,0,0;0.0;31.465654068747885;26.9191467740004;26.28482026848514
1;0,2,0,2;29.315083246115393;30.85967688016068;28.237153342637573;0.0
2;0,0,2,2;28.657980531401954;0.0;29.300089975361765;30.612445916335
3;2,0,0,2;29.307012062855954;29.25859113811383;28.59376732648875;31.468805139022386
4;0,2,2,0;28.855932552441022;31.218279534849756;28.152230173579934;29.538881587145653
5;2,0,2,0;26.2396816447543;25.86272150450492;0.0;31.870140204435806
6;4,0,0,2;18.096422119948237;21.524150639931975;24.67407151978731;29.75600496408652
```

Figure 8: parent Q-table representation

5.3.1.2 Child:

```
parent_index;operations;direction
60;mh;2
60;rr;3
59;rr;3
59;mv;2
```

Figure 9: child Q-table representation

In theory the splitting of parent to child reduces the storage space needed for the Q-table. A normal Q-table often sits at around 5 GB depending on the complexity of the game and the number of possible actions and states. This idea should cut the memory usage in half.

One issue that could arise from this approach can be the loss of information as not every state abstraction can be done to the corresponding actions. The child inherits the actions of the parent state-action pair. If the actions of the parent state result in loss of information, in this case the wrong direction, the child will transform the actions of the parent to match the best direction saved.

5.3.1.3 Splitting

A parent state-action pair consists of the parent state and the actions for the given state.

$$p_{sa} = (p, q_p)$$

With:

$$q_p = (Q_p^{up}, Q_p^{down}, Q_p^{left}, Q_p^{right})$$

q_p Q-values for parent state-action pair

Q_p Q-value for parent

$$c = (p, O, d_c)$$

- $p \in P$ the parent state for the child
- O Operational sequence
- $d_c \in A$ the saved best direction for the child state

Example:

$$p_{sa} = (1; 0,2,2,0; 25.34,12.23,29.123,26.296)$$
$$c_{sa} = (0,4,4,0; 25.34,21.56,25.23,22.896; 0)$$

$$c = O(p) + direction_c$$

$$c = (1; d; 0)$$

5.3.1.4 Reconstruction

The reconstructed state of the child is generated with the operational sequence saved in the child plus the parent state.

$$c = O(p)$$

Additionally the transformation must also be applied to the actions of the parent:

$$T_O: \mathbb{R}^4 \rightarrow \mathbb{R}^4$$

$$q'_c = T_O(q_p)$$

q'_c the reconstructed actions from the parent state-action pair to the child

If the best direction of the child and the best direction of the parent is not the same the actions will be transformed to fit the best direction.

$$\hat{d}_c = arg \max_{a \in A} q'_c(a)$$

If the best direction of the child and the best direction of the parent is the same the actions of the parent are applied to the child.

$$\hat{d}_c = d_c$$
$$q_c = q'_c$$

Example:

$$s = O(c)$$
$$s = (0,4,4,0;0;25.34,12.23,29.123,26.296;0)$$

If $bestDirection \neq direction_s$:

$$apply\ best\ direction\ (s)$$
$$s = (0,4,4,0;0;25.34,12.23,25.33,25.34;0)$$

6 Environment

The environment in this thesis is the game 2048. The environment consists of 3 versions. A 2 by 2 grid version a 3 by 3 version and the normal 4 by 4 version. As previously mentioned the environments where chosen to test different results and conditions. The total amount of states found in the 2 by 2 grid is 476. This was found after 10 hours of continuous running of the program. The total state space of the 3 by 3 version and the 4 by 4 version has not yet been reached and takes up to much storage space for this thesis. The current state space is 295.330 for the 3 by 3 version and 2.554.721 for the 4 by 4 version.

The idea behind the different environment versions is to make it easier to test code and find anomalies or issues. The choice of 2048 as the environment makes this particularly suitable for this thesis. The idea is to split the total number of states into child and parent states but as this would take a lot of compute power and RAM the states are counted in batches. The batch size determines the number of states compared with each other. A batch count of 10.000 would compare 10.000 states with each other to determine similarities. A problem arises though if the states are not sorted though. Therefore before the splitting of parent and child states can occure the list of states is sorted from smallest total value to highest total value where a state with a lot of zeros has a smaller value than a state with no zero. After the list is sorted the batch processing can occur.

The size of the processed children and parents can be found in the child or parent Q-table. For the 2 by 2 version of the environment the total number of child states is 120 and the total number of parent states the total number is 357.

7 Python Implementation

7.1 Main

The main module is the entry point of the application. It coordinates the interaction between the agent the environment and the play function. The user can determine what kind of agent they want and what size the environment should have. Later after the agent trained the user can call the play module to play with the provided child and parent Q-tabels. The play mode also provides evaluation.

7.2 Environment

The environment is the game logic of the game 2048. It represents an interface for the agent to interact with the game. It is implemented as a class that manages the grid, applies actions and returns rewards for each chosen action.
The environment contains the grid on which the game is played. The user can determine the grid size by using the "–grid" or "-g" flag when running the train module in the main.py script.
The Grid is initialized with zeros and two tiles are spawned at the beginning of the game followed with 1 tile after every legal action.

7.2.1 Core Interaction

The main function of the environment is to give the agent the next state the reward and a terminal condition. This is done via the step() function. It receives as an argument from the agent a action a. This can be up, down, left and right.
It merges the tiles in the corresponding direction and calculates the reward based on the merged tiles. The legal actions are determined via the function find_available_actions() which checks all tiles to see if a given direction is allowed or not.
If a move was valid the step() function creates a new random tile at a random position and returns the state to the agent together with the reward gained from the chosen action.

7.2.2 Reward Function

The reward is based on the value of all merged tiles. If a move was deemed ineffective or bad a penalty is applied.
This reward is calculated though following function:

$$r_t = value\ of\ tiles\ merged\ in\ row\ or\ column - 1 + 0.1 * the\ number\ of\ new\ empty\ tiles$$

The new empty tiles are counted as a bonus as this encourages open board states which makes it easier for the parent child splitting to occur.

7.3 Agent

The agent implements a value-based reinforcement learning approach. It uses a Q-table and learns a mapping from state to action values like a normal value-based reinforcement learning model. The difference is that the state action pairs are not saved as a singular Q-table but saved as a parent and child state Q-table. The learning process itself is still the classical idea of storing and updating values for states on a singular Q-table.

In each step, the agent observes the current grid finds a set of valid actions, selects one of these actions according to its exploration strategy, and updates its stored action values based on the reward returned by the environment. After a certain amount of states determined by a user chosen batch size the singular Q-table is split into child and parent state. To determine if a child state is a child of a parent state the agent looks through every state and compares them with each other. If two states are equivalent under the allowed matrix operations, one state can be stored explicitly as a parent while the other is represented as a child together with the transformation path and the saved best direction.

The agent therefore has two roles. It acts as the learning component of the reinforcement learning loop and manages the abstraction pipeline. It provides the functionality to load and save the child and parent Q-tables divide the Q-table used for the learning path of the agent and reconstructs the children and parent states to a singular reconstructed Q-table for continued training.

7.3.1 Q-Table Representation

The Q-table is implemented as a normal dictionary to save time. The lookup time of a python dictionary is O(1) which helps in the finding of children. Since every step requires the agent to read and write action value pairs for a given state the fast lookup is essential for the training efficiency. The states are saved as a flattened tuple and used as the key for the lookup.

$$\begin{bmatrix} 0 & 2 \\ 2 & 4 \end{bmatrix}$$

Is transformed into the tuple:

$$(0, 2, 2, 4)$$

The reason the states are saved as tuple is so they can be directly used as keys in python and tuples reduce the number of storage space needed.

The value is the list of actions for that state. If a state is encountered for the first time during the training a new directory entry will be created automatically. The action values would be initialized as 0 in this case. After every loop the action values will get updated by the learning rule as the agent trains.

In addition the agent also works with 2 additional Q-tables the parent Q-table which is a smaller version of the original Q-table and a child Q-table which only saves the connection to the parent the best direction and the operations to be applied to the parent.

7.3.2 Action Selection

The agent uses a simple ε-greedy strategy. This strategy is the most common strategy in value-based learning models. It balances exploration and exploitation. Exploration is as the name suggests the ability of the agent to choose new paths and find new states and exploitation is when the agent recalls previous states in his Q-table and chooses the best action from memory.
Action selection is limited by the number of allowed actions provided by the environment. Through find_available_actions() the agent is limited to only possible actions which reduces decision wasting time and ensures compatibility between the learning process and the game.
The agent decides between exploitation and exploration by choosing a random number between 0 and 1 and then comparing it to ε. If the random number is greater or equal to ε the agent performs exploitation if not the agent performs exploration.
If the agent is in exploitation mode the agent searches through the Q-table until finding the matching states and compares their action-values. It chooses the highest value and performs the action. If the state is not yet stored it falls back to a random action-value.

This procedure can be summarized as the following:

1. with probability ε, choose a random valid action
2. with probability $1 - \varepsilon$, choose the valid action with the highest Q-value
3. if the state is unknown, choose a random valid action
4. if several valid actions share the same maximum value, choose randomly among them

7.3.3 Learning Process

The learning process of the agent is based on the standard Q-learning principle. After each loop with the environment the agent updates the Q-values of the selected action in the state.

The Q-values are updated using the Q-learning update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a)$$

But in the agent no explicit discount factor is used so the actual Q-learning update rule is:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha(r + \max_{a'} Q(s',a'))$$

This means that the value of the next state is added without additional discounting. As result the future rewards are treated the same as immediate rewards. Before the update is done the program ensures that both states, the current and the next state exist in the Q-table. If not the agent initializes the action-values as 0.0.

7.3.4 Training Loop

The training loop defines the repeated interaction between the agent and the environment over a predefined number of episodes chosen by the user. It is used to gradually improve the agents Q-values.

At the beginning of each new episode the agent resets the environment with the environment function reset() and a new initial state is returned. The agent then interacts with the environment via actions until the episode ends. For each step the environment sends the agent all possible actions. The agent then selects an action via the previously described action selection utilizing the ε – greedy strategy. The agent interacts with the environment via the step() function in which it provides the selected action and the environment returns the next state and the reward given that action.

The agent then updates its Q-table via the previously discussed learning process. After a from the user decided amount of episodes the Q-table can be saved periodically to ensure no data loss during outages. The exploration rate ε is reduced and the next episode starts. In this thesis the Q-table is separated into parent and child Q-tables at the end of every episode but in theory it can be done at any point. To save RAM space and CPU usage the Q-table can be separated into batches on which the program then runs the separation.

7.3.5 Q-Table Storage

This thesis supports the idea of splitting the existing Q-table into two to save storage space and make transfer of data easier for bigger projects. In a normal RL context the Q-table is a singular model file most often a python pickle file or a csv file in which the Q-table is saved as a state-action pair. As previously mentioned in the context of the game 2048 this would be a tuple for the state and a list filled with the weight of each possible actions.

The idea of this thesis is as already mentioned the concept of saving this large singular Q-table as a combination of two Q-tables. The parent Q-table is a representation of all states. It consists of root states, actions and the index of the state. A root state is simply the state from which the children states can be build from. In theory this cuts the total numbers of states saved in half has most states should be children of root states.

The child Q-table stores the transformation mappings that need to be applied to the parent in order to achieve the original Q-table. It consists of parent id, operations and best direction.

The last table in this program is the testing table or reconstructed table. Was the name already implies it is the original Q-table reconstructed from the parent and child table. In theory the reconstructed Q-table and the original Q-table should be the same. This can only happen if the loss of information is not large enough to cause problems.

7.4 Operations

The core contribution of this thesis is the abstraction of states using transformation operations. These operations exploit structural similarities between states such as scaling of values or rotation of states. This results one representative parent state and many child states. In context of this thesis a operation must not only be able to modify the state but also the action values of the state. Therefore each operation consists of two linked parts. State transformation which uses simple matrix operations to transform an existing state to another state and and a corresponding action transformation which reorders the action values.

7.4.1 Transformation Operations

The implementation defines a set of 4 transformation operations but can be increased to also include the previously mentioned operations in 5.2 . These operations are stored in the Pre-Calculator as a mapping between state transformation and the corresponding action transformation. The currently active implementation includes:

Rotation (r):
The rotation operation applies 90 degrees counterclockwise rotation to the state matrix. This is possible due to the grid based environment of the game 2048. Rotation exploits geometrical symmetries in the game board.

Doubling (d):
The doubling operation multiplies all tile values of the state by a factor of two. A board containing values such as 2, 4 or 8 is therefore structurally similar to a board containing 4, 8 and 16.

Mirroring horizontally (mh):

The horizontal mirror operation swaps the left and right sides of the grid.

Mirroring vertically (mv):
The vertical mirror operation swaps the upper and lower sides of the grid.

7.4.2 Action Transformation

When a state is transformed the action values corresponding to that state must also be transformed in order to not loss information. If the state is rotated or mirrored the direction associated with each action must be transformed as well. Otherwise the reconstructed child would inherit the incorrect action preferences from its parent.

For this reason every state transformation in the implementation is paired with a action transformation.
For example, in a 90 degrees rotation the action transformation is so that the action rotate in the same way:

- new up = old right
- new down = old left
- new left = old up
- new right = old down

For a mirroring on the horizontal axis the left and right actions are exchanged and for a mirroring on the vertical axis the up and down actions are exchanged. Scaling or doubling does not change the structure of the grid and must therefore not be applied to the actions.

7.4.3 PreCalculator

The PreCalculator is the python script that includes the previously mentioned operations. It is called in order to split an existing Q-table into child and parent. The PreCalculator is responsible for exploring the transformation space of a given parent state. It is given a parent state and determines all possible reachable child states. It then saves all operations and child states in a list for later comparison. If a state matches with one of these calculated child states the operations are saved to that child state and the child is recognized as a child of the parent. If a given state is not a possible child of a parent state the state becomes a parent itself and the PreCalculator is called again. The new calculated states are appended to the existing list of reachable states.

This implementation is made with a breadth-first search strategy. The search begins with the original parent state at depth 0 and an empty operation path. After each visited node the PreCalculator applies the available operations and inserts the resulting states into the search

queue for the next state to check against. The process continues until the maximum depth is reached.

Each generated result contains the following:

- The transformed state
- The transformed action values
- The sequence of operations used to reach that state

7.4.4 Parent-Child Abstraction

The parent child abstraction is the practical application of the operations described above. Its purpose is to split the original Q-table into the parent and the child table.

During this process the original Q-table is read and for each state the program checks whether its already in the reachable list calculated by the PreCalculator or if it's a parent. This lookup is supported by a SQLite based index in order to reduce memory usage during this process as without it the program can run through 64GB of RAM really quickly.

If the state is found in the reachable index it becomes a child. It inherits the index of the parent the sequence of operations and the best direction of the original child. The best direction is simply the highest action in the original action values of the child.

If the state is not found in the reachable list it becomes a new parent and the PreCalculator is calculating new child states reachable from the parent. This procedure allows the abstraction processto replace many state action entries. Instead of storing full Q-values for every state only root actions are stored. This results in less total storage requirements for the learned Q-table.

7.4.5 Reconstruction

To evaluate the abstraction the parent and child tables are merged back into one reconstructed Q-table. This table is then compared to the original Q-table. The reconstruction begins with the table of parent states. It is loaded into memory as the parent states already contain full state-action information. Then the program runs through each child state and applies the operations in order to return the children to complete states. For each child the parent state is loaded and the operations are applied. Then the actions of the parent are applied to the child and the action transformation takes place. If the best direction stored in the child does not match with the actions of the parents the actions will be minimally adjusted until the best direction of the child matches. This is done via small value offsets ensuring that the original action structure is changed as little as possible.

The reconstructed Q-table is then saved for later comparison.

8 Concept

9 Evaluation

9.1 Setup

9.2 Metrics

9.3 Compression Results

9.4 Reconstruction Error

9.5 Policy Accuracy

9.6 Runtime and Memory Overhead

10 Bibliography

[1] C. J. Watkins und P. Dayan, „Q-Learning," Kluwer Academic Publisher, Netherlands, 1992.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, Second edition in progress, 2. ed., Cambridge, MA, USA: MIT Press, 2018, [Online: https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf]: A Bradford Book, 2014, 2015.

[3] V. Mnih, K. Kavukcuoglu, D. Silver und et al., „Nature," 2015. [Online]. Available: https://doi.org/10.1038/nature14236.

[4] T. Yang, C. Jin, Z. Wang, M. Wang und M. I. Jordan, „On Function Approximation in Reinforcement Learning: Optimism in the Face of Large State Spaces," Princeton, California, Northwestern, 2021.

[5] M. Szubert und . J. Wojciech, „Temporal Difference Learning of N-Tuple Networks for the Game 2048," in *IEEE Conference on Computational Intelligence and Games, CIG*, Institute of Computing Science, Poznan University of Technology, Poznan, Poland, 2014.

[6] L. P. Kaelbling, M. L. Littman und A. W. Moore, „Reinforcement Learning: A Survey," Journal of Artificial Intelligence research 4, 1996.

[7] S. J. Kazemitabar und H. Beigy, „Automatic Discovery of Subgoals in Reinforcement Learning Using Strongly Connected Components," in *Advances in Neuro-Information Processing*, Berlin, Heidelberg, 2009.

[8] K. Gupta und H. Najaran, „Exploiting Abstract Symmetries in Reinforcement Learning for Complex Environments," in *2022 International Conference on Robotics and Automation (ICRA)*, Philadelphia, PA, USA, 2022.

[9] D. Abel, D. Arumugam, L. Lehnert und M. L. Littman, „State Abstraction for Lifelong Reinforcement Learning".

[10] L. Li, T. J. Walsh und M. L. Littman, „Towards a Unified Theory of State Abstraction for MDPs," Department of Computer Science Rutgers university, Piscataway.

List of Figures

Figure 1: Example player input right tile shift 10
Figure 2: Environment Agent loop 13
Figure 3: normal Q-table representation 15
Figure 4: Transformation-Based Rotation 18
Figure 5: Transformation-Based Scaling 19
Figure 6: Transformation-Based edge expansion 19
Figure 7: Transformation-Based Mirroring 20
Figure 8: parent Q-table representation 21
Figure 9: child Q-table representation.....21

List of Tables

Es konnten keine Einträge für ein Abbildungsverzeichnis gefunden werden.

List of Abbreviations

ML	Machine Learning
RL	Reinforcement Learning

Documentation table of AI-based tools

AI-based tools	Intended use	Prompt, source, page, paragraph...
DeepL Translate	Translation of an article in English	Source (XXX), Chapter X on page X-X
ChatGPT (4.0)	Grammar and spelling	"Please list issues with spelling and grammar in the following text: ..." Entire document

A: Heading of Appendix A

B: Heading of Appendix B